Day 70: The Information Bottleneck — Where One Fixed-Size State Stops Being Enough

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 1 | Complete study guide

Watch: Attention: Problems with Seq2seq Models (Greg Durrett, UT Austin) | S18 Sequence to Sequence models: Attention Models (Carnegie Mellon University Deep Learning)

Script: `day70_information_bottleneck.py` · every number in this guide is copied by program from `day70_run_output.txt`, the real output of that script.

Day 69's encoder-decoder copied 3 to 8 digits almost perfectly. Trained on 3 to 20 digits it fails on the long ones: exact match on 20-digit copies is 0.004 with hidden 32 and 0.157 with hidden 64. A controlled test shows why: when every input has 20 digits and only the first k must be written back, the model is perfect for k = 4 (1.000) and fails for k = 20, although the distance to the first digit is the same in both. What breaks is **how much** must pass through one fixed-size state, not how far back it is. That is the information bottleneck, and it is what attention (Day 71) removes.

0. Where today fits

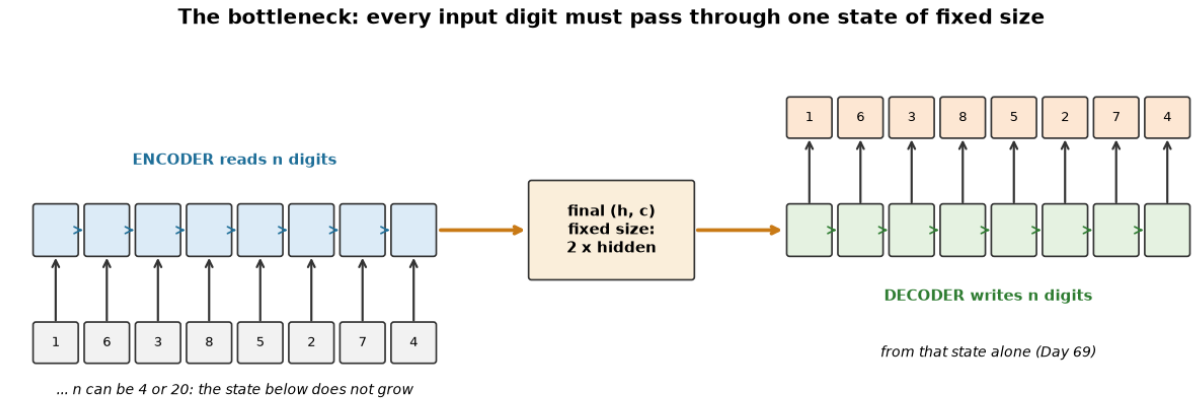
DEFINITION — start here, no prior knowledge assumed

In a basic encoder-decoder, everything the decoder knows about the input comes from the encoder's final state (h, c), whose size is fixed (2 × hidden numbers) whatever the input's length. A fixed-size channel that all the information must pass through is called a **bottleneck**.

REAL-WORLD SCENARIO

Being told a phone number once and repeating it later: 4 digits are easy, 20 are not, even though you heard the first digit of both the same few seconds ago. Your short-term memory, not the delay, is the limit.

SYSTEM DIAGRAM



However long the input, only the final (h, c) reaches the decoder.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Day 69's model on longer sequences

DEFINITION — start here, no prior knowledge assumed

Day 69's COPY model, trained exactly as there on 3 to 8 digits, tested on 4 to 16 digits.

REAL-WORLD SCENARIO

A student who practised only 8-digit phone numbers, asked to repeat a 16-digit one.

SPECIFICATION

Model	Hidden 32, lengths 3–8, 20,000 steps, seed 0 (Day 69's training run, repeated).
Test	300 fixed sequences per length.

Why this test

The obvious first question; its answer is ambiguous, which is why the rest of today exists.

IMPLEMENTATION

```
net, _ = train_cached("copy", 32, 0, 20000, 3, 8, eval_lengths=(8,))
print(f"\n{'length':>7}" + "".join(f"{n:>7}" for n in range(4, 17, 2)))
rows = [exact_and_token(net, fixed_test(n, "copy")) for n in range(4, 17, 2)]
```

ACTUAL OUTPUT

```
 length      4      6      8     10     12     14     16
  exact  1.000  1.000  0.947  0.113  0.000  0.000  0.000
  token  1.000  1.000  0.988  0.801  0.626  0.532  0.435
Actual output from this lesson's run.
```

What it means

Exact match falls from 0.947 at 8 digits to 0.113 at 10 and 0 from 12 on. But this mixes two problems: the model never saw these lengths, and it must hold more digits. Sections 2 and 3 separate them.

WHY CHOOSE THIS?

A quick, intuitive first look.

WHY NOT THIS? (TRADEOFFS)

Confounded: it cannot tell a capacity limit from never having seen the length.

2. Trained on every length it is tested on

DEFINITION — start here, no prior knowledge assumed

The fair version: train on 3 to 20 digits, test on lengths inside that range, at three state sizes. Exact match is tracked during training to see whether the runs have finished learning.

REAL-WORLD SCENARIO

Practising phone numbers of every length up to 20, then being tested on them.

MATHEMATICS

state handed to the decoder: 2H numbers (h and c); information in n digits: $n \times \log_2 8 = 3n$ *bits*

20 digits = 60 bits; even H = 16 gives 32 real numbers, far more than 60 bits in principle

SPECIFICATION

Runs	COPY, lengths 3–20, hidden 16, 32, 64, seeds 0, 1, 2, 40,000 steps, plain SGD lr 0.5, clip 1.0, batch 32.
Tracking	Exact match at 4, 8, 12, 16, 20 digits every 5,000 steps.

Why this test

It removes the never-seen-this-length excuse, and the three sizes show whether a bigger state helps.

IMPLEMENTATION

```
def train(task, hidden, seed, steps, lo, hi, k=None, every=EVERY, eval_lengths=EVAL_LENGTHS):
    net = Seq2Seq(hidden=hidden, seed=seed)
    rng = np.random.RandomState(seed + 100)
    tests = {n: fixed_test(n, task, k) for n in eval_lengths}
    curve = []
    for step in range(1, steps + 1):
        n = rng.randint(lo, hi + 1)
        if k is None:
            Xs, Xd, Y, _ = make_batch(rng, 32, n, task)
        else:
            Xs, Xd, Y, _ = make_batch_first(rng, 32, n, min(k, n))
        net.train_step(Xs, Xd, Y)
        if step % every == 0:
            curve.append([exact_and_token(net, tests[n])[0] for n in eval_lengths])
    return net, np.array(curve)
```

ACTUAL OUTPUT

```
 hidden  state size     n=4     n=8    n=12    n=16    n=20   seeds
     16          32   0.972   0.556   0.069   0.000   0.000       3
     32          64   1.000   0.949   0.691   0.198   0.004       3
     64         128   1.000   0.994   0.952   0.742   0.157       3
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
is training finished? exact match at n=12 and n=16 over the last checkpoints (mean of seeds):
 hidden   n    @20k    @25k    @30k    @35k    @40k
     16  12   0.026   0.024   0.053   0.054   0.069
     16  16   0.000   0.000   0.000   0.000   0.000
     32  12   0.446   0.622   0.584   0.517   0.691
     32  16   0.028   0.097   0.098   0.094   0.198
     64  12   0.737   0.916   0.843   0.881   0.952
     64  16   0.377   0.539   0.558   0.589   0.742
Actual output from this lesson's run.
```

What it means

Every size fails as sequences get longer, and a bigger state fails later: at 12 digits exact match is 0.069, 0.691 and 0.952 for hidden 16, 32 and 64; at 20 digits 0.000, 0.004 and 0.157. Counting bits, even the smallest state could store 20 digits in principle; what fails is *learning* to squeeze them in and read them back out with gradient descent.

Training was not finished for hidden 32 and 64: at 16 digits hidden 64 rose from 0.377 at 20,000 steps to 0.742 at 40,000. Section 6 trains three times longer to see how far that goes.

WHY CHOOSE THIS?

A fair test: every length tested was trained on, three sizes, three seeds, tracked over time.

WHY NOT THIS? (TRADEOFFS)

Still a budget-limited result for the larger sizes; Section 6 checks it.

3. Too far, or too much?

DEFINITION — start here, no prior knowledge assumed

The key control. Every input has exactly 20 digits; the model must write back only the **first k**. For every k, the first digit to write was read 20 steps before the encoder's final state, so the **distance** is always the same. Only the **amount** to hold changes.

REAL-WORLD SCENARIO

Hearing a 20-digit number and being asked for only its first 4 digits, or its first 12, or all 20.

SPECIFICATION

Runs	Hidden 32, k = 4, 8, 12, 16, 20, inputs always 20 digits, 30,000 steps, three seeds.
Note	k = 20 is full COPY of 20 digits, trained only on 20-digit inputs.

Why this test

Long sequences are hard in two ways at once: early digits are far back, and there are many of them. This holds the first fixed and varies the second.

IMPLEMENTATION

```
def make_batch_first(rng, m, n, k):
    src = rng.randint(1, 9, size=(m, n))
    tgt = src[:, :k]
    dec_in = np.concatenate([np.full((m, 1), GO), tgt], axis=1)
    dec_out = np.concatenate([tgt, np.full((m, 1), EOS)], axis=1)
    return np.eye(V)[src], np.eye(V)[dec_in], dec_out, src
```

ACTUAL OUTPUT

```
   k  exact match  token acc              per seed (exact)
   4        1.000      1.000           1.000  1.000  1.000
   8        0.986      0.995           0.967  0.993  0.997
  12        0.761      0.927           0.587  0.797  0.900
  16        0.106      0.594           0.023  0.190  0.103
  20        0.013      0.447           0.000  0.040  0.000
Actual output from this lesson's run.
```

What it means

The amount, not the distance. Holding 4 digits across 20 steps is perfect (1.000 for every seed); 8 is nearly so (0.986); then exact match falls steeply: 0.761 for 12, 0.106 for 16 and 0.013 for all 20. The LSTM has no trouble keeping information for 20 steps (Day 65's forget gate at work); it cannot keep this much of it in one state.

WHY CHOOSE THIS?

Isolates the cause: the same distance with different amounts.

WHY NOT THIS? (TRADEOFFS)

One state size and one budget; larger states move the curve, as Section 2 suggests.

4. Where the errors are

DEFINITION — start here, no prior knowledge assumed

Token accuracy at each output position of the 20-digit COPY test, for the hidden-32 models of Section 2.

REAL-WORLD SCENARIO

Which digits of a long number someone gets wrong when repeating it.

SPECIFICATION

Data	300 fixed 20-digit sequences; three seeds; position 1 is the first digit written.

Why this test

Day 69 found errors gathering at the end of the output; this checks the same at the length where the model fails.

IMPLEMENTATION

```
acc = np.mean([(train_cached("copy", 32, s, STEPS_MAIN, 3, 20)[0].generate(Xs, 21) ==
    Y).mean(axis=0)
               for s in SEEDS], axis=0)
```

ACTUAL OUTPUT

```
 position     1     3     5     7     9    11    13    15    17    19
 accuracy 0.984 0.969 0.927 0.850 0.643 0.608 0.530 0.396 0.270 0.262
guessing a digit: 0.125
Actual output from this lesson's run.
```

What it means

The first digits are written almost perfectly (0.984 at position 1) and accuracy falls steadily along the output, to 0.262 at position 19, still above guessing (0.125). The model holds the beginning of the sequence and gets progressively vaguer about the rest, as Day 69's shorter sequences hinted.

WHY CHOOSE THIS?

Shows the failure is gradual, not all-or-nothing.

WHY NOT THIS? (TRADEOFFS)

Token accuracy hides how often whole sequences fail (Section 2).

5. What the final state holds

DEFINITION — start here, no prior knowledge assumed

A **linear probe**: a simple classifier trained to read one input digit from the encoder's final (h, c) alone. One probe per input position. If a probe reads its digit well, that digit is stored in the state in an easy-to-read form.

REAL-WORLD SCENARIO

Checking what someone actually remembers of a long number by asking for each digit separately.

MATHEMATICS

$$\text{probe for position } p: \quad p(\text{digit}) = \text{softmax}(\, z\, W_p + b_p \,), \quad z = \text{standardised } [h\,;\,c]$$

SPECIFICATION

Encoders	The hidden-32 COPY model (seed 0), the same model untrained, and the k = 4 model of Section 3 (seed 0).
Data	20-digit inputs; 3,000 training and 1,000 test sequences per probe.

Why this test

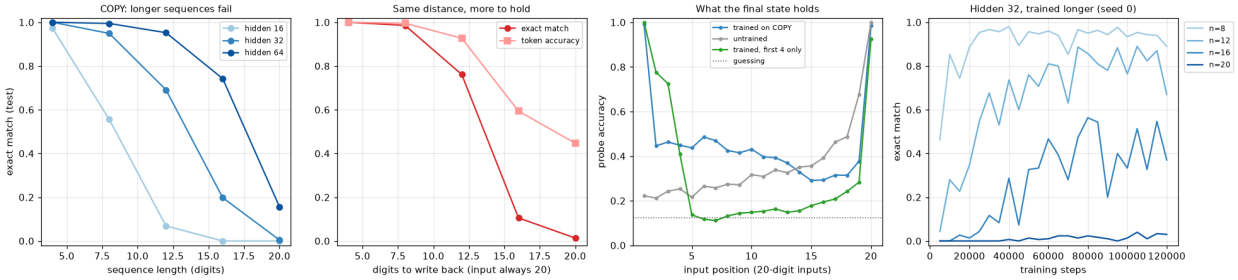
It looks directly at the bottleneck: which digits make it into the state.

IMPLEMENTATION

```
def fit_probe(F, y, iters=400, lr=0.5):
    mu, sd = F.mean(axis=0), F.std(axis=0) + 1e-8
    Z = (F - mu) / sd
    W = np.zeros((F.shape[1], V)); b = np.zeros(V)
    for _ in range(iters):
        p = softmax(Z @ W + b)
        p[np.arange(len(y)), y] -= 1
        W -= lr * Z.T @ p / len(y); b -= lr * p.mean(axis=0)
    return lambda G: ((G - mu) / sd @ W + b).argmax(axis=1)
```

ACTUAL OUTPUT

```
input position              1     3     5     7     9    11    13    15    17    19
trained on COPY         0.990 0.463 0.437 0.469 0.415 0.397 0.369 0.291 0.315 0.377
untrained               0.222 0.243 0.216 0.258 0.272 0.309 0.326 0.356 0.463 0.677
trained, first 4 only   1.000 0.725 0.137 0.111 0.144 0.153 0.148 0.178 0.208 0.283
guessing: 0.125
Actual output from this lesson's run.
```



Left to right: exact match by length; the same-distance test; the probe at every input position; hidden 32 trained three times longer.

What it means

The model trained to write back 4 digits keeps exactly what it needs. Its probe reads position 1 perfectly (1.000) and position 3 well (0.725), and from position 5 on the digits are close to guessing (0.137, 0.111). The state is spent on the first digits and the rest is let go.

The COPY model spreads itself thin. It keeps position 1 (0.990) but the middle positions only partly (between 0.291 and 0.469 on the printed positions). Near the end it holds the digits less readably than an untrained encoder (0.377 at position 19 against 0.677), although the plot shows every encoder still holds the very last digit well. It has traded some of an LSTM's natural recency for a share of the state for every digit, and no middle digit is kept well.

WHY CHOOSE THIS?

A direct view of what passes through the bottleneck.

WHY NOT THIS? (TRADEOFFS)

A linear probe only finds what is easy to read; information stored in a more tangled form would be missed.

6. Is it just training time?

DEFINITION — start here, no prior knowledge assumed

Section 2's hidden-32 COPY run (seed 0) continued to 120,000 steps, three times the budget, with exact match recorded every 5,000 steps.

REAL-WORLD SCENARIO

Practising the 20-digit number three times as long.

SPECIFICATION

Run	Hidden 32, COPY, lengths 3–20, seed 0, 120,000 steps (its first 40,000 steps are Section 2's run).

Why this test

Section 2 showed the curves still rising; this tests whether more time closes the gap.

IMPLEMENTATION

```
_, curve = train_cached("copy", 32, 0, STEPS_LONG, 3, 20)
cols = [EVAL_LENGTHS.index(n) for n in (8, 12, 16)]
print(f"\n{'steps':>8}{'n=8':>8}{'n=12':>8}{'n=16':>8}")
for i in range(3, len(curve), 4):
    print(f"{(i + 1) * EVERY:>8}" + "".join(f"{curve[i, c]:>8.3f}" for c in cols))
```

ACTUAL OUTPUT

```
   steps     n=8    n=12    n=16
   20000   0.887   0.347   0.013
   40000   0.980   0.737   0.287
   60000   0.960   0.810   0.467
   80000   0.950   0.857   0.563
  100000   0.933   0.763   0.333
  120000   0.890   0.670   0.370
Actual output from this lesson's run.
```

What it means

More training helps up to a point and then stops helping: 16-digit exact match goes from 0.287 at 40,000 steps to 0.563 at 80,000, then falls back to 0.370 at 120,000; 8-digit and 12-digit accuracy fall too. The right panel of the figure shows the run swinging up and down rather than converging. Within three times the budget the bottleneck model did not get close to solving 16 or 20 digits.

WHY CHOOSE THIS?

Directly tests the most likely objection.

WHY NOT THIS? (TRADEOFFS)

One seed and one learning rate; a smaller learning rate late in training might hold its peak.

7. Does reversing help?

DEFINITION — start here, no prior knowledge assumed

Sutskever et al. (2014) found that reversing the input made seq2seq training easier. Here, REVERSE instead of COPY at hidden 32, everything else as Section 2.

REAL-WORLD SCENARIO

Repeating a number backwards: the first digit you say is the one you just heard.

SPECIFICATION

Runs	REVERSE, lengths 3–20, hidden 32, seeds 0, 1, 2, 40,000 steps.

Why this test

Reversing shortens the distance for the first outputs, but the last outputs then need the digits read longest ago.

IMPLEMENTATION

```
copy = np.mean([train_cached("copy", 32, s, STEPS_MAIN, 3, 20)[1][-1] for s in SEEDS], axis=0)
rev = np.mean([train_cached("reverse", 32, s, STEPS_MAIN, 3, 20)[1][-1] for s in SEEDS], axis=0)
```

ACTUAL OUTPUT

```
    task     n=4     n=8    n=12    n=16    n=20
    copy   1.000   0.949   0.691   0.198   0.004
 reverse   1.000   0.989   0.889   0.447   0.057
Actual output from this lesson's run.
```

What it means

Reversing helps at every length (0.889 against 0.691 at 12 digits, 0.447 against 0.198 at 16), as Sutskever et al. found, but it does not remove the bottleneck: at 20 digits it reaches only 0.057.

WHY CHOOSE THIS?

A cheap trick that helps.

WHY NOT THIS? (TRADEOFFS)

It shifts the problem; the amount that must pass through one state is unchanged.

Interview preparation — technical questions

Q: What is the information bottleneck in a basic seq2seq model?

A: The decoder receives only the encoder's final state, a fixed-size vector, however long the input is.

Q: How would you show that a model fails because of capacity and not distance?

A: Fix the distance and vary the amount: here, 20-digit inputs with k digits to write back; k = 4 is perfect, k = 20 fails.

Q: What is a linear probe?

A: A simple classifier trained on a frozen representation to test what information it holds in an easy-to-read form.

Q: Your seq2seq accuracy still rises at the end of training. What do you do?

A: Train longer and track it: here three times the budget helped up to 80,000 steps and then fell back.

Q: Why does a larger hidden size help but not fix the problem?

A: It enlarges the bottleneck; the amount to hold still grows with the input.

Interview preparation — theoretical questions

Q: Could a 64-number state hold 20 digits?

A: In principle yes (60 bits); the failure is in learning such a compression with gradient descent, not in raw capacity.

Q: Why did reversing the input help?

A: The first outputs then depend on the most recently read inputs, which shortens the dependencies the model learns first.

Q: What is the standard fix for the bottleneck?

A: Attention: let the decoder read all encoder states at every step (Day 71).

Q: Why did errors gather at the end of the output?

A: The probe shows the state keeps the first digits best; later digits are held only partly.

Key takeaway and what's next

KEY TAKEAWAY

One fixed-size state limits how much of the input the decoder can get, and the limit is about amount, not distance: holding 4 digits across 20 steps is perfect, holding 20 is not. Bigger states, three times the training and reversing the input all help a little; none removes the limit. The probe shows the state being spent: on just what is needed when the task allows it, spread too thin when it does not.

- Bottleneck: the decoder sees only the encoder's final (h, c), whatever the input length.
- 20-digit COPY exact match: 0.000 / 0.004 / 0.157 at hidden 16 / 32 / 64.
- Same distance, more to hold: k = 4 gives 1.000, k = 20 gives 0.013.

- Errors grow along the output; the state keeps the first digits best.
- Three times the training: a peak, then back down. Reversing: helps, does not fix.
- Controls matter: Section 1 alone could not tell capacity from unfamiliar lengths.

Suggested watch order, if starting from zero

- 1. Attention: Problems with Seq2seq Models (Greg Durrett, UT Austin) — the bottleneck, briefly, and why attention follows.
- 2. S18 Sequence to Sequence models: Attention Models (Carnegie Mellon University Deep Learning) — a full lecture from sequence-to-sequence to attention.

Day 71 preview

- **Additive (Bahdanau) attention.** Keep every encoder state and let the decoder choose, at each output step, which ones to read. Day 71 builds it from scratch and runs exactly today's Section 2 test.
- Today's exercise script, `day70_exercise.py`, isolates the read that replaces the bottleneck: a softmax-weighted average of memory slots, with padding masked out.